AdaptiveSched: Self-Tuning CPU Scheduler Selection via Online Bandit Learning and SLO-Aware Rollback

ARJUN TYAGI (Reg No 24BCE0605) and ARITRA GHOSH (24BCE0598)
SCOPE Department, VIT VELLORE

Abstract—General-purpose operating systems traditionally depend on a single CPU scheduler, which fails to deliver optimal performance in the face of ever-changing workloads like gaming, compilation and network request handling. While the introduction of the sched_ext framework in Linux provides the ability to dynamically load different schedulers, it does not offer an automated way to determine which scheduler to pick. All existing adaptive approaches depend on offline trained machine learning models and hard-wired, hardware-specific calibration tables, thus suffering from poor adaptivity to new environments. In order to fill in these gaps, we propose a user-space scheduling agent, which treats scheduler switching as an online multi-armed bandit problem. With the use of continuously collected latency and throughput measurements as a reward signal, the agent automatically selects the optimal sched_ext scheduler without any offline training. Moreover, in order to ensure the stability of the system, our solution is equipped with an automated watchdog, which evaluates the post-switch performance with respect to the Service Level Objective (SLO) set by the user and automatically switches back to the default scheduler in case of degradation. This paper introduces the architecture of the proposed system and the methodology for its evaluation on the real Linux kernel in different workloads.

Index Terms—Linux kernel, CPU scheduling, multi-armed bandits, reinforcement learning, sched_ext, service level objectives.

I. Literature Review

The problem of optimizing CPU scheduling in the presence of diverse workloads has attracted much attention in research, including everything from delegating scheduling to user space to machine learning techniques. While conventional operating systems use static default scheduler, the recent developments include dynamic tuning of parameters as well as entire policy switching.

One of the solutions to this problem is to delegate scheduling policies to the user-space. The work done by Humphries et al. introduces ghOSt framework, which allows complex scheduling policies to be defined in user-space agents and provides a lot of flexibility in data center workloads [1]. Another example is Syrup, which enables developers to define custom scheduling policies on multiple execution levels and shows a lot of performance improvement [2]. However, even though these solutions provide means for custom policies, they require humans to manually configure them for particular applications.

Machine learning, especially reinforcement learning (RL) technique has been used more and more often to automatize system optimization process. For example, Han et al. show how to use policy gradient RL for dynamically tuning parameters of Linux CPU frequency governors on Android smartphones to adapt to changing workload [3]. Pattanayak uses online RL technique to optimize the Linux CPU-idle menu governor based on the statistics of load balancing in the scheduler [4]. These works prove applicability of online learning techniques to kernel subsystems, but focus only on fine-tuning parameters of particular governor or scheduler.

Within the context of cluster scheduling, there is an example of Decima system, which uses deep reinforcement learning to learn workload-specific scheduling algorithms [5]. Even though this work shows great results for data processing clusters, it requires lots of offline training and operates on a higher level of abstraction than OS-level CPU scheduling. More recently, Tang et al. show how to optimize Linux scheduling based on global runqueues using the SCX framework [6].

The most similar to our proposed solution is the line of work that uses trained models for scheduler selection. Recently published preprints describe ASA system, which uses predictions from models along with time-weighted voting and pre-configured, machine-specific mapping tables to automatically select different schedulers using sched_ext framework [7]. This work proves feasibility of dynamic scheduler switching in the real kernel, but relies on the offline training and static calibration tables.

Our work closes the gaps by viewing the problem of schedulers selection as non-stationary online multi-armed bandit problem. Contrary to existing works which rely on offline calibration or tuning of parameters within single policy, our approach requires no prior knowledge and learns the optimal scheduler based on the real-time latency and throughput measurements. Also, in order to address the safety issue with policy switching absent in existing literature, we introduce post-switch watchdog that guarantees adherence to the Service Level Objective (SLO) of the user by enforcing the automatic rollback in case of performance degradation.

This work is a project proposal and literature review for academic purposes.

References

[1] J. Humphries et al., "ghOSt: Fast and Flexible User-Space Delegation of Linux Scheduling," in Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP), 2021, pp. 588-604.

[2] K. Kaffes et al., "Syrup: User-Defined Scheduling Across the Stack," in Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP), 2021, pp. 605-620.

[3] C. Han et al., "Performance Improvement of Linux CPU Scheduler Using Policy Gradient Reinforcement Learning for Android Smartphones," IEEE Access, vol. 8, pp. 15994-16003, 2020.

[4] A. Pattanayak, "Linux CPU-Idle Menu Governor with Online Reinforcement Learning and Scheduler Load Balancing Statistics," in IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), 2019, pp. 1-6.

[5] H. Mao et al., "Learning Scheduling Algorithms for Data Processing Clusters," in Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM), 2019, pp. 270-288.

[6] J. Tang, Y. Gao, L. Li, and C. Lin, "Optimizing Linux Scheduling Based on Global Runqueue with SCX," in IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2024, pp. 1-6.

[7] Anonymous Authors, "Mixture-of-Schedulers: Adaptive Scheduling Agent," arXiv preprint arXiv:2511.11628, 2025.